VibeRobot!: Vibe-to-Verify End-User Programming for Robot Arms

Anonymous Author(s)

Abstract

Natural-language task authoring is making robot programming accessible to non-experts, but practical success still breaks at the same point: after first-generation plans fail. In our iterative development of **VibeRobot**, users repeatedly hit a failure pattern of opaque errors, weak context carryover across retries, and unclear control boundaries between preview and execution. We present **ArmCraft**, a *Vibe-to-Verify* interaction design that treats robot vibe coding as a verification loop rather than a one-shot generation problem. ArmCraft combines (1) task-spec normalization from natural language, (2) simulation-first gating with explicit user approval before movement, (3) novice-oriented failure cards that expose where and why plans fail, and (4) constrained patch suggestions for low-friction repair. We contribute an implementation-grounded architecture, a sharpened research agenda on performance/learning/attribution, and a CHI-oriented mixed-method evaluation plan. The core claim is that better robot end-user development requires UX for debugging, safety, and accountability, not only stronger planners.

CCS Concepts

• **Human-centered computing** → **Human computer interaction (HCI)**; **Empirical studies in HCI**; • **Computing methodologies** → *Robotic planning*.

Keywords

robot end-user development, LLM, simulation-first verification, interactive debugging, human-AI collaboration, responsibility attribution

ACM Reference Format:

Anonymous Author(s). 2026. VibeRobot!: Vibe-to-Verify End-User Programming for Robot Arms. In *CHI '27: Proceedings of the 2027 CHI Conference on Human Factors in Computing Systems.* ACM, New York, NY, USA, 5 pages. https://doi.org/TBD

1 Introduction

Language-first robot interfaces are improving rapidly through LLM-based planning and control systems such as SayCan, Inner Monologue, ProgPrompt, Code as Policies, VoxPoser, PaLM-E, and RT-2 [2, 8, 9, 14, 15, 21, 24]. Quantitatively, this line already reports large-scale capability signals (e.g., RT-2 evaluation over 6k trials, and PaLM-E instantiated at 562B parameters) [8, 9]. In parallel, early end-user systems now show that non-experts can produce useful robot apps with LLM support [11, 16]. This shift makes "natural language to robot behavior" feasible, but not yet reliable in novice workflows.

Our formative QA and engineering triage in VibeRobot indicate a consistent bottleneck: generation is easy; *failure recovery* is hard. Users can issue follow-up corrections (e.g., "not that one," "do it properly"), but systems may lose context, produce under-specified plans, or execute with weak transparency. This motivates three practical risks we explicitly design for: low success under iteration, weak safety confidence, and unstable responsibility attribution when outcomes fail.

We frame this as an HCI systems problem, not only a model-quality problem. ArmCraft therefore proposes a **Vibe-to-Verify** loop: users can ask in plain language, but every execution must pass simulation preview and explicit approval; failures are translated into novice-readable diagnosis; patch suggestions are small, actionable, and immediately testable.

1.1 Contributions

This paper contributes:

- **C1 (Design)**: ArmCraft, an interaction architecture for robot end-user development that integrates sim-first approval, failure inspection, and patch-based repair.
- **C2 (Implementation)**: A running prototype that operationalizes ArmCraft with end-to-end stages from intent extraction to approval-gated execution and iterative retries.
- **C3 (Research Agenda)**: A CHI-oriented evaluation design with explicit hypotheses across productivity, transfer learning, explanation uptake, trust calibration, and responsibility attribution.
- **C4 (Methodological Bar)**: A reproducibility plan (structured logs, analysis preregistration, artifact release) aimed at rigorous systems+user-study evidence.

2 Related Work

2.1 LLM-Based Robot Planning and Embodied Control

Recent work demonstrates broad capability gains in language-conditioned robot behavior: affordance-aware skill selection [2], explicit language-mediated reasoning loops [15], code-generation-based policy construction [21], language-to-plan synthesis [24], multimodal embodied models [8, 9], and compositional spatial value maps [14]. Code as Policies also reports a 39.8% HumanEval score under hierarchical code generation, showing stronger program synthesis capacity behind embodied control logic [21]. However, these systems primarily optimize planning and execution quality, while user-facing debugging and accountability workflows remain under-specified.

2.2 Robot End-User Development

Longstanding robot EUD literature emphasizes teaching aids, demonstration interfaces, and low-code authoring for non-programmers [3, 4, 10]. Newer LLM-mediated systems (e.g., Alchemist [16], Cocobo [11]) reduce authoring friction further. Yet, a persistent gap is *iterative failure repair*: novice users still need support to interpret

CHI '27, Yokohama, Japan
2026. ACM ISBN TBD
https://doi.org/TBD

Table 1: Selected quantitative signals from prior work that motivate ArmCraft.

Work	Quantitative signal
RT-2	Reports evaluation over 6k robotic trials [8].
PaLM-E	Instantiates an embodied multimodal model at 562B parameters [9].
Code as Policies	Reports 39.8% on HumanEval via hierarchical code generation [21].
HRI trust meta-analysis	Synthesizes 29 studies with aggregate effects $\bar{r} = 0.26$, $\bar{d} = 0.71$ [12].

failure causes, choose safe corrections, and verify improvements before real motion.

2.3 Actionable Explanations and Explanatory Debugging

HCI and IUI research has established that explanations are useful only when they support user action [17, 18]. Broader XAI work in HCI similarly emphasizes explanation utility, accountability, and user experience [1, 22]. ArmCraft adapts this line to embodied task execution by binding explanations directly to corrective patches and immediate re-simulation.

2.4 Trust Calibration and Responsibility Attribution

Human factors work shows that trust in automation should be calibrated to system capability, not blindly increased [12, 20, 23]. A quantitative anchor is Hancock et al.'s HRI trust meta-analysis: 29 studies, with aggregate effect sizes of $\bar{r} = 0.26$ (correlational) and $\bar{d} = 0.71$ (experimental) [12]. In robot vibe coding, trust calibration and attribution are tightly coupled: users decide whether failure is "my instruction," "the robot," or "the AI planner." Existing robot programming interfaces rarely expose this attribution dynamic as a primary design objective.

2.5 Gap Summary

The literature strongly covers language-to-robot generation and general explainability principles. The open CHI opportunity is the **interaction loop from failure to verified repair** for novices, with measurable effects on learning transfer, control, and attribution. ArmCraft targets that gap.

3 ArmCraft: Vibe-to-Verify Architecture

Figure 1 shows the full architecture as implemented in our prototype and extended for CHI evaluation.

3.1 Design Requirements

ArmCraft is driven by three requirements derived from repeated QA and user-facing failures:

- **R1: Controllability**. No robot motion without explicit approval after preview.
- **R2: Repairability**. Failures must be translated into reasons and concrete next actions.
- **R3: Accountability**. System transitions and safety constraints must be inspectable and auditable.

These requirements are aligned with prior human-AI interaction guidance on user control, feedback, and trust calibration [5, 20].

3.2 Five Fixed Modules

A. Task Spec Builder. Natural language is normalized into explicit slots (goal, targets, constraints, success criteria, assumptions) to avoid downstream ambiguity.

B. Task-to-Code Agent. The planner emits *code + test intent + execution constraints* rather than code alone, enabling verification and post-hoc analysis.

C. Sim-First Runner. UI transition is strictly `Simulate -> Approve & Execute`. Pre-approval simulation is visible in chat preview; the execution view remains non-moving until approval.

D. Failure Inspector. Failures are summarized as cards with type, time slice, and causal cues (e.g., collision overlap, unstable grasp, unreachable pose, workspace violation).

E. Patch Suggestions. The system proposes 2–3 bounded edits (e.g., top-down regrasp, speed cap, path replan, target lock) to reduce decision fatigue and accelerate retry.

3.3 Loop-Level Interaction

Figure 3 summarizes the user loop.

3.4 Pipeline Hardening from QA

We incorporated concrete hardening updates motivated by observed failure cases:

- **Semantic-to-executable normalization**: repairs underspecified action parameters before execution.
- **Retry-context target lock**: keeps object references stable across corrective follow-ups.
- **Scene isolation**: deep-copy scene presets to prevent cross-session state contamination.
- **Approval-state enforcement**: simulation and execution channels remain explicitly separated until user consent.

4 Research Questions and Hypotheses

RQ1 (Performance and Learning). Does ArmCraft increase novice task success and transfer to perturbed scenes?

RQ2 (Comprehension-to-Action). Which failure representations most effectively drive correct next-step actions?

RQ3 (Attribution and Trust). How does ArmCraft shift responsibility attribution and trust calibration under repeated failure-repair cycles?

We operationalize these with directional hypotheses:

- **H1**: ArmCraft reduces time-to-first-success and iterations vs. Vibe-only and Sim-first-only baselines.
- **H2**: ArmCraft improves transfer success under object-position perturbations.
- **H3**: ArmCraft reduces workload and increases perceived control without inflating blind trust.

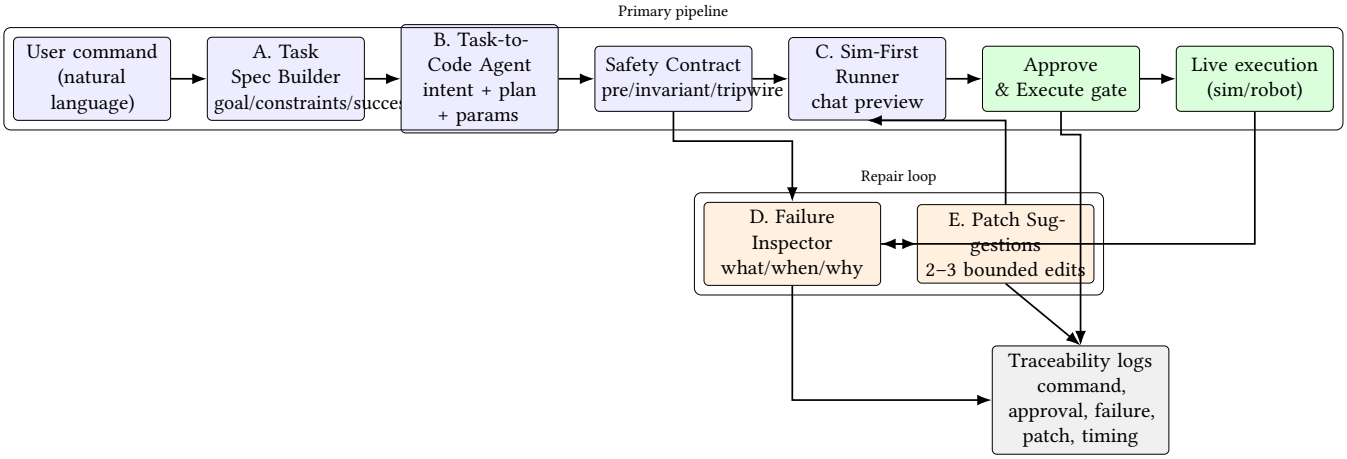

Figure 1: ArmCraft architecture mapped to the running prototype. Execution is blocked until simulation preview and explicit user approval succeed.

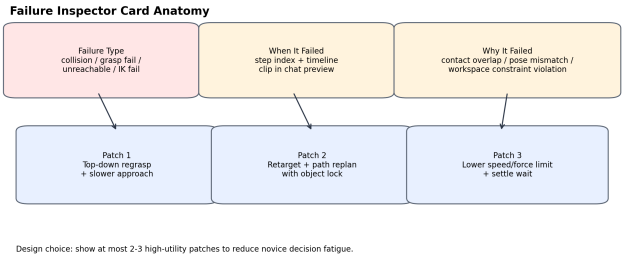

Figure 2: Failure Inspector structure in ArmCraft: what failed, when it failed, why it failed, and a short patch set for immediate re-simulation.

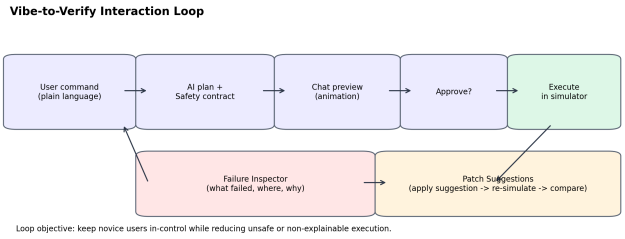

Figure 3: Vibe-to-Verify interaction loop. The user remains in control through explicit approval and patch-guided repair.

- **H4**: ArmCraft produces more calibrated attribution (less default self-blame or agent-blame) by exposing failure causality.

5 Evaluation Plan

Figure 4 outlines the two-study program.

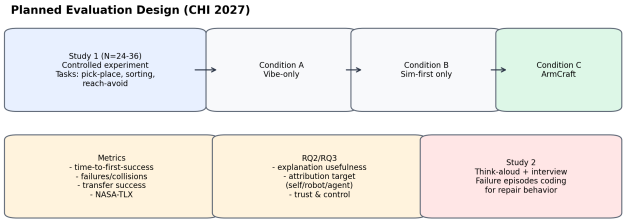

Figure 4: Planned CHI evaluation: controlled experiment (Study 1) plus qualitative deep dive (Study 2).

Table 2: Condition features in Study 1.

Feature	Vibe-only	Sim-first	ArmCraft
Sim preview before execute	No	Yes	Yes
Explicit approval gate	No	Yes	Yes
Failure card diagnosis	No	No	Yes
Patch suggestions (2–3)	No	No	Yes
Retry context lock	No	No	Yes

5.1 Study 1: Controlled Experiment (N=24–36)

Conditions.

- **Vibe-only**: natural language plan generation + direct simulation execution.
- **Sim-first only**: forced preview/approval but raw logs (no Failure Inspector/Patches).
- **ArmCraft**: full loop with inspector and patch suggestions.

Tasks. Pick-and-place, sorting, and reach-avoid tasks under standardized scenes and perturbation trials.

5.2 Measures

Primary outcomes: time-to-first-success, success rate, iteration count, collision count, safety-constraint violations, transfer success.

Secondary outcomes: NASA-TLX workload [13], trust and perceived control scales [12, 20], and post-failure attribution choices (self/robot/AI/planner+justification).

Process traces: event logs, stage durations, rejected vs. accepted patches, and post-patch delta in failure type.

5.3 Analysis Plan

For Study 1, we will use mixed-effects models with condition as fixed effect and participant/task as random effects, following established guidance for confirmatory modeling of repeated-measures data [6]. For non-normal outcomes, we will use generalized mixed models. Planned post-hoc contrasts are ArmCraft vs each baseline with multiplicity control. We will report effect sizes and confidence intervals, not only p-values [19].

5.4 Study 2: Think-Aloud + Semi-Structured Interviews

Study 2 examines *how* users interpret failures and choose patches. We will code:

- confusion and recovery episodes,
- explanation uptake patterns,
- attribution shifts across retries,
- trust and control narratives after success/failure.

Two coders will independently code transcripts and resolve disagreements through adjudication, using thematic analysis procedures common in HCI qualitative work [7].

6 Rigor, Reproducibility, and Ethics

6.1 Rigor Plan

To support CHI-level rigor, we plan preregistered hypotheses/analysis, fixed task scripts, and release of anonymized logs plus analysis code at submission-ready artifact freeze.

6.2 Claim Traceability

We separate evidence types for clarity:

- **Literature-backed claims**: all background and theory claims are cited to prior peer-reviewed work.
- **Artifact-backed claims**: system-behavior claims are tied to concrete implementation and regression tests in the artifact (e.g., approval gating, retry target lock, scene isolation, safety contract generation).
- **Future-effect claims**: efficacy claims are stated as hypotheses (H1–H4) and not presented as achieved results.

This is intended to minimize over-claiming and make each claim auditable during review. In our submission package, this is operationalized as a claim-evidence matrix that maps major statements to citations, code paths, and regression tests.

6.3 Operational Safety

All executions are approval-gated and constrained by explicit safety contracts (workspace, speed limits, collision tripwires, emergency stops). Study protocols include stop controls and non-deceptive consent disclosures.

6.4 External Validity Limits

Our current evidence target is simulation-grounded interaction quality. Real hardware transfer remains a follow-up stage and may introduce perception/calibration failures beyond the current simulator envelope.

7 Discussion

ArmCraft argues that robot vibe coding should be evaluated as an *interactive debugging system*, not only as a planner. This reframes the unit of success from "one generated plan" to "human+AI recovery trajectory under uncertainty." If confirmed empirically, this has design implications for future robot authoring tools:

- make approval boundaries explicit and inspectable,
- keep explanations tightly coupled to edits users can actually perform,
- and treat attribution calibration as a first-class UX target.

8 Conclusion

We presented an upgraded CHI 2027 paper plan for ArmCraft, a Vibe-to-Verify interaction design for robot arm end-user development. The system-level novelty is the integration of sim-first approval, novice-readable failure inspection, and bounded patch-guided repair in a single loop. The empirical agenda explicitly tests not only task success but also learning transfer, explanation utility, and responsibility attribution. Our objective is to make language-first robot programming safer, more controllable, and more learnable for non-experts.

References

[1] Ashraf Abdul, Jo Vermeulen, Danding Wang, Brian Y. Lim, and Mohan Kankanhalli. 2018. Trends and Trajectories for Explainable, Accountable and Intelligible Systems: An HCI Research Agenda. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–18. doi:10.1145/3173574.3174156

[2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Ruben Ruano, Pierre Sermanet, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sicheng Xu, Andy Z. Lee, Johnny Lee, Lisa Tseng, and Vincent Vanhoucke. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691 [cs.RO] https://arxiv.org/abs/2204.01691

[3] Gopika Ajaykumar, Maia Stiber, and Chien-Ming Huang. 2021. Designing User-Centric Programming Aids for Kinesthetic Teaching of Collaborative Robots. *Robotics and Autonomous Systems* 145 (2021), 103845. doi:10.1016/j.robot.2021.103845

[4] Sonya Alexandrova, Maya Cakmak, Kaijen Hsiao, and Leila Takayama. 2014. Robot Programming by Demonstration with Interactive Action Visualizations. In *Robotics: Science and Systems X*. Robotics: Science and Systems Foundation. doi:10.15607/RSS.2014.X.048

[5] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–13. doi:10.1145/3290605.3300233

[6] Dale J. Barr, Roger Levy, Christoph Scheepers, and Harry J. Tily. 2013. Random Effects Structure for Confirmatory Hypothesis Testing: Keep It Maximal. *Journal of Memory and Language* 68, 3 (2013), 255–278. doi:10.1016/j.jml.2012.11.001

[7] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kelvin Han, Karol Hausman, Alex Herzog, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Andy Z. Lee, Kristen Lee, Tsang-Wei E. Lee, Yao Lu, Carolina Parada, Karl Pertsch, Kanishka Rao, Pierre Sermanet, Tianhe Zhang, Ted Xiao, Peng Xu, Ted Xu, Fei Xia, Jie Xiao, Wenhao Yu, Brianna Zitkovich, Andy Zeng, Sergey Levine, and Vincent Vanhoucke. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818 [cs.RO] https://arxiv.org/abs/2307.15818

[9] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. 2023. PaLM-E: An Embodied Multimodal Language Model. arXiv:2303.03378 [cs.RO] https://arxiv.org/abs/2303.03378

[10] Daniela Fogli, Luigi Gargioni, Giovanni Guida, and Fabio Tampalini. 2022. A Hybrid Approach to User-Oriented Programming of Collaborative Robots. *Robotics and Computer-Integrated Manufacturing* 73 (2022), 102234. doi:10.1016/j.rcim.2021.102234

[11] Yate Ge, Yi Dai, Run Shan, Kechun Li, Yuanda Hu, and Xiaohua Sun. 2024. Cocobo: Exploring Large Language Models as the Engine for End-User Robot Programming. arXiv:2407.20712 [cs.HC] https://arxiv.org/abs/2407.20712

[12] Peter A. Hancock, Deborah R. Billings, Kristin E. Schaefer, Jessie Y. C. Chen, Ewart J. de Visser, and Raja Parasuraman. 2011. A Meta-Analysis of Factors Affecting Trust in Human-Robot Interaction. *Human Factors* 53, 5 (2011), 517–527. doi:10.1177/0018720811417254

[13] Sandra G. Hart and Lowell E. Staveland. 1988. *Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research.* Elsevier, 139–183. doi:10.1016/S0166-4115(08)62386-9

[14] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. 2023. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. arXiv:2307.05973 [cs.RO] https://arxiv.org/abs/2307.05973

[15] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. 2022. Inner Monologue: Embodied Reasoning through Planning with Language Models. arXiv:2207.05608 [cs.RO] https://arxiv.org/abs/2207.05608

[16] Ulas Berk Karli, Juo-Tung Chen, Victor Nikhil Antony, and Chien-Ming Huang. 2024. Alchemist: LLM-Aided End-User Development of Robot Applications. In *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction.* ACM, New York, NY, USA, 361–370. doi:10.1145/3610977.3634969

[17] Todd Kulesza, Margaret Burnett, Weng-Keen Wong, and Simone Stumpf. 2015. Principles of Explanatory Debugging to Personalize Interactive Machine Learning. In *Proceedings of the 20th International Conference on Intelligent User Interfaces.* ACM, New York, NY, USA, 126–137. doi:10.1145/2678025.2701399

[18] Todd Kulesza, Simone Stumpf, Margaret Burnett, Sherry Yang, Irwin Kwan, and Weng-Keen Wong. 2013. Too Much, Too Little, or Just Right? Ways Explanations Impact End Users' Mental Models. In *2013 IEEE Symposium on Visual Languages and Human Centric Computing.* IEEE, 3–10. doi:10.1109/VLHCC.2013.6645235

[19] Daniël Lakens. 2013. Calculating and Reporting Effect Sizes to Facilitate Cumulative Science: A Practical Primer for t-tests and ANOVAs. *Frontiers in Psychology* 4 (2013), 863. doi:10.3389/fpsyg.2013.00863

[20] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80. doi:10.1518/hfes.46.1.50.30392

[21] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as Policies: Language Model Programs for Embodied Control. arXiv:2209.07753 [cs.RO] https://arxiv.org/abs/2209.07753

[22] Q. Vera Liao, Daniel Gruen, and Sarah Miller. 2020. Questioning the AI: Informing Design Practices for Explainable AI User Experiences. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems.* ACM, New York, NY, USA, 1–15. doi:10.1145/3313831.3376590

[23] Raja Parasuraman and Victor Riley. 1997. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors* 39, 2 (1997), 230–253. doi:10.1518/001872097778543886

[24] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2022. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. arXiv:2209.11302 [cs.RO] https://arxiv.org/abs/2209.11302